

K-Means Clustering

The background is a dark navy blue. On the left, there is a large, light purple circle. On the right, there is a large, dark grey circle. Scattered around these are several smaller circles in purple, orange, and teal. Some of these smaller circles have a white outline, making them stand out. The overall aesthetic is modern and tech-oriented.

A Visual Guide with Real-World Examples

From Customer Segments to Fraud Detection

Adila Abdulwahid • Financial Data Scientist

What is K-Means?

K-Means is an **unsupervised** machine learning algorithm that groups data into **K clusters** based on similarity — without needing labeled data.



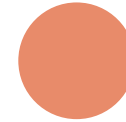
Unsupervised

No labels needed.
It finds natural groupings on its own.



Distance-Based

Groups points that are close together (Euclidean distance).



Iterative

Repeats until cluster centers stop moving (convergence).

How K-Means Works

1

Choose K

Decide how many clusters (K) you want. This is your only input.

2

Place Centroids

Randomly place K centroid points in the feature space.

3

Assign Points

Every data point joins its nearest centroid (by Euclidean distance).

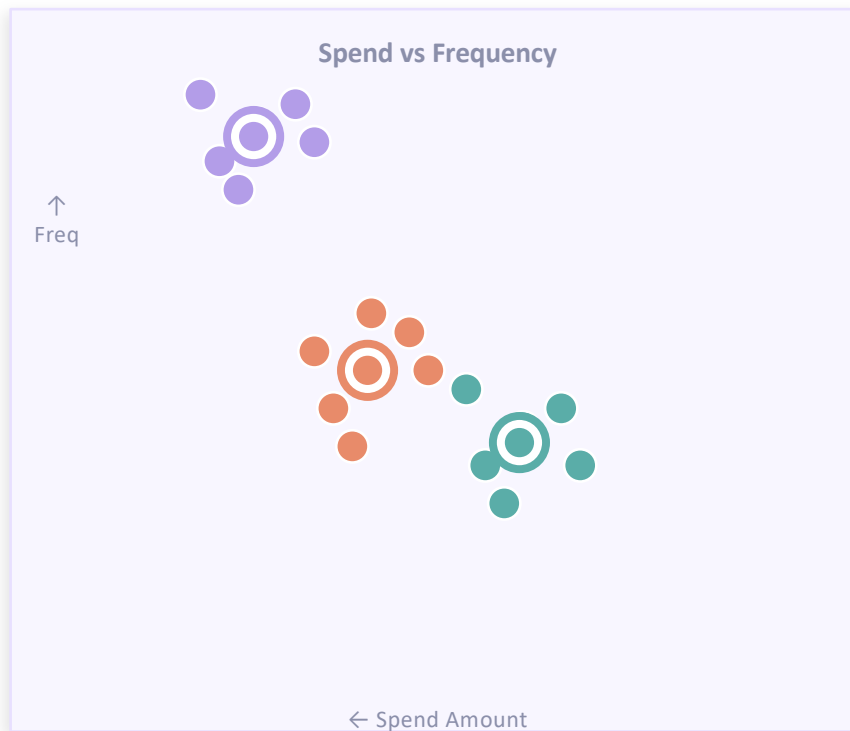
4

Move Centroids

Recalculate each centroid as the mean of its assigned points. Repeat.

Repeat steps 3 & 4 until centroids no longer move — this is called convergence.

Customer Spend Segmentation



Premium Loyalists

High avg spend: \$850/mo

15+ transactions/mo

→ Offer exclusive rewards

Occasional Shoppers

Low avg spend: \$120/mo

2–3 transactions/mo

→ Run activation campaigns

Mid-Tier Regular

Mid spend: \$340/mo

7–10 transactions/mo

→ Upsell & loyalty program

EXAMPLE 2 — BANKING

Bank Fraud Detection

Feature	Normal Cluster	Suspicious	High-Risk Fraud
Transaction Amount	< \$200	\$200–\$2,000	> \$5,000
Time of Day	9am – 8pm	8pm – midnight	Midnight – 4am
Distance from Home	< 5 km	50–200 km	> 500 km
Transactions / 24h	1–3	4–8	10+
Account Age	> 1 year	3–12 months	< 30 days



How a new transaction gets classified:

Measure the Euclidean distance from the new transaction to every cluster centroid → assign it to the nearest one → if distance > threshold, flag as anomaly.

Patient Risk Cohort Discovery

Cohort A — Low Risk

- Age: 25–40
- 0–1 chronic conditions
- 0 ER visits / year

→ Annual wellness reminders

Cohort B — Moderate Risk

- Age: 45–60
- 2–3 chronic conditions
- 1–2 ER visits / year

→ Quarterly check-ins

Cohort C — High Risk

- Age: 60–75
- 4+ chronic conditions
- 3–5 ER visits / year

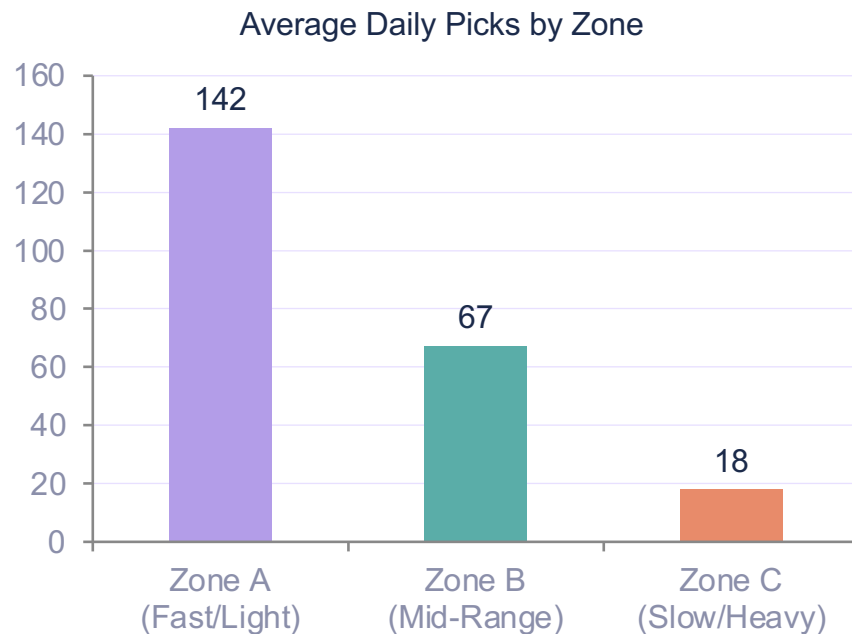
→ Care management program

Cohort D — Critical

- Age: 70+
- 5+ conditions + polypharmacy
- 6+ ER visits / year

→ Dedicated case manager

Warehouse Zone Optimization



Zone A — Front of Warehouse

Fast-moving, lightweight SKUs
Picked 100–200x per day
→ Place nearest the shipping dock

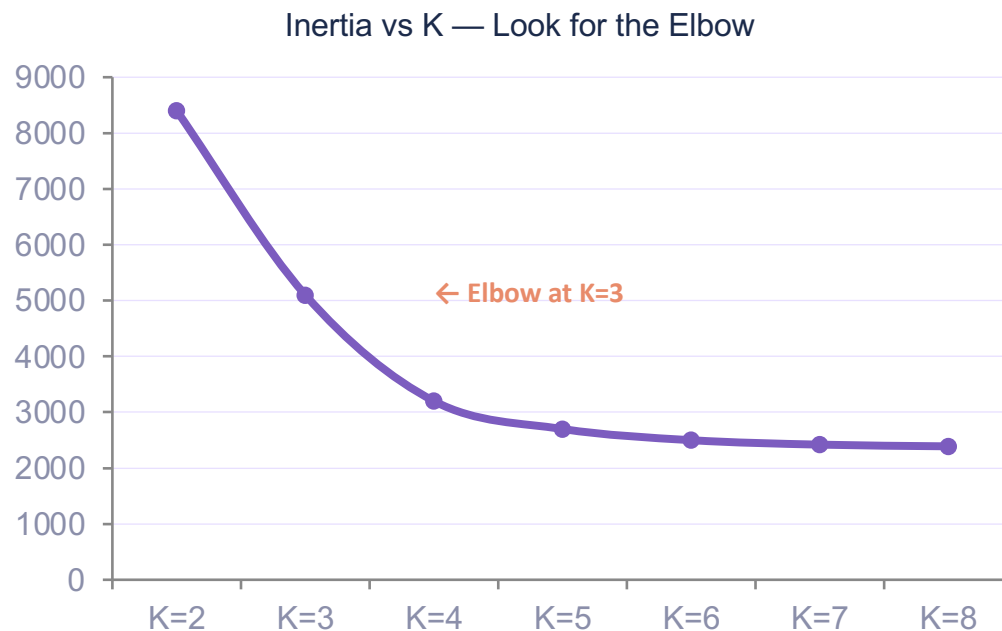
Zone B — Middle Section

Mid-frequency, mid-weight SKUs
Picked 40–100x per day
→ Second priority placement

Zone C — Back of Warehouse

Slow-moving, heavy SKUs
Picked < 30x per day
→ Minimize picker travel distance

Choosing K — The Elbow Method



What is Inertia?

The sum of squared distances from each data point to its cluster centroid. Lower = tighter clusters.

The Elbow Rule

Adding more clusters stops significantly reducing inertia — that 'bend' is your optimal K.

Silhouette Score

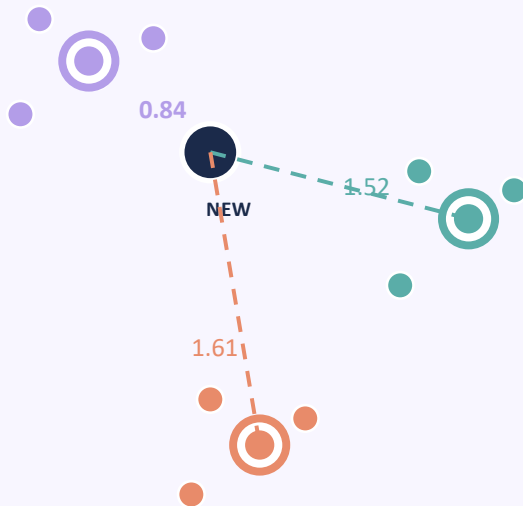
A second check: measures how similar a point is to its own cluster vs others. Higher is better (max 1.0).

Business Sense Check

Always validate K against domain knowledge. K=3 in fraud (Normal/Suspicious/High-Risk) just makes sense.

How to Classify a New Data Point

Distance to Each Centroid



✓ Nearest centroid: Cluster A (0.84) → Assign to Cluster A

1

Scale the new point

Apply the SAME StandardScaler used during training — never re-fit.

2

Calculate distances

Compute Euclidean distance from the new point to every centroid.

3

Assign to nearest

Pick the centroid with the smallest distance. That's the cluster.

4

Check for anomalies

If distance > threshold (e.g. 2.5), flag as anomaly regardless of cluster.

Strengths & Limitations

✓ Strengths

- ✓ Simple and fast — scales to large datasets
- ✓ No labeled data needed (unsupervised)
- ✓ Easy to explain: 'group by similarity'
- ✓ Feature importance via cluster profiles
- ✓ Works well for customer segmentation, anomaly detection

✗ Limitations

- ✗ You must choose K in advance
- ✗ Sensitive to outliers — they pull centroids
- ✗ Assumes spherical, equal-sized clusters
- ✗ Doesn't give probabilities — only assignments
- ✗ Results change if you re-run (random init)

Tip: Pair K-Means with a supervised model once you have confirmed labels for even better results.

The Essential K-Means Code

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from scipy.spatial.distance import cdist

# 1. Scale features (ALWAYS required for K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. Find optimal K with Elbow + Silhouette
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)

# 3. Fit final model & score new transactions
new_scaled = scaler.transform(new_data) # same scaler!
distances = cdist(new_scaled, km.cluster_centers_)
cluster = np.argmin(distances) # nearest centroid
```

Full code + manager explainer in the project PDF ↗

Key Takeaways

1

Unsupervised — no labels needed

K-Means finds natural groups in raw data. Perfect when you don't have fraud labels.

2

Scale your features first

StandardScaler is non-negotiable. Reuse the SAME scaler when scoring new points.

3

Distance = confidence

The closer a new point is to the fraud centroid, the higher the risk. Distance > threshold = anomaly.

4

Explain with analogies

Tell your manager: 'We grouped transactions into neighborhoods. New ones are compared to find their closest match.'